Assignment No: 7

Title: Apply JS frontend functionalities and form validations.

Aim: To implement client-side frontend functionality in JavaScript and to validate an HTML form using both the built-in Constraint Validation API and custom JavaScript rules including regular expressions.

Objectives:

1. To understand the need for client-side validation and its relationship to server-side validation.
2. To apply HTML5 built-in validation attributes: required, type, minlength, maxlength, pattern, min and max.
3. To write custom validation functions using regular expressions.
4. To use the Constraint Validation API (checkValidity, setCustomValidity, validity).
5. To display inline, field-level error messages instead of alert boxes.
6. To validate a file input for type and size before the file is processed.

Hardware Used:

- Computer or Laptop
- Keyboard
- Mouse
- Minimum 4 GB RAM

Software Used:

- Visual Studio Code (VS Code)
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)
- Operating System: Windows / Linux / macOS

Theory:

1. Why Client-Side Validation

Client-side validation gives the user instant feedback and prevents obviously invalid data from being sent to the server. It is a usability feature, not a security feature: a user can disable JavaScript or send a request directly, so every rule must also be enforced on the server.

2. HTML5 Built-in Validation Attributes

Attribute	Meaning
required	The field must not be empty.
type="email" / "url" / "number"	The browser checks the value format.
minlength / maxlength	Minimum and maximum number of characters.
min / max	Numeric or date bounds.
pattern	A regular expression the value must match.

accept	Restricts the file types offered by a file picker.

3. The Constraint Validation API

Member	Purpose
element.checkValidity()	Returns true if the field satisfies its constraints.
element.validity	An object of flags: valueMissing, typeMismatch, patternMismatch, tooShort, rangeOverflow, customError.
element.setCustomValidity(msg)	Marks the field invalid with a custom message; pass "" to clear it.
form.noValidate	Disables the browser's default bubbles so custom messages can be rendered inline.

4. Regular Expressions for Validation

Pattern	Validates
^[A-Za-z]{3,40}$	A name of 3 to 40 letters and spaces
^[^@\s]+@[^@\s]+\.[A-Za-z]{2,}$	A basic email address
^[6-9]\d{9}$	A 10-digit Indian mobile number
^LS-\d{4}$	A LipiSutra accession number such as LS-0147
^(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9]).{8,}$	A password of 8+ characters with an uppercase letter, a digit and a symbol

5. Validating a File Input

The accept attribute only filters the picker dialog; it does not stop a user from dropping any file. The file object exposed by the input must therefore be checked in JavaScript for its MIME type (file.type) and its size in bytes (file.size) before it is read.

6. Frontend Functionality Beyond Validation

- Live character counters driven by the input event.
- Enabling or disabling the submit button from the overall form validity.
- Showing a preview of a selected image with URL.createObjectURL().
- Debouncing a search field so work runs only after the user pauses typing.
- Keeping the entered values after a failed submit instead of clearing the form.

Course Project Implementation:

Course-Project Mapping: Not available in the course project in a submittable form. LipiSutra does contain form controls - the historian editor in src/components/HistorianView.jsx uses controlled <input> and <textarea> elements with min={0} / max={100} and a disabled state once a document is locked, and src/components/PublicView.jsx has a search field with a keydown handler - but there is no validation logic anywhere: no required fields, no pattern checks, no error messages and no submit guard. A complete validated form is therefore supplied below, modelled on a LipiSutra manuscript-submission screen.

Requirement	In project?	Where / gap
Controlled form inputs	Yes	src/components/HistorianView.jsx (renderInput)
Numeric bounds (min / max)	Partially	min={0} max={100} on the accuracy field
Keyboard event on a field	Yes	PublicView.jsx handleSearch (Enter key)
required / pattern / error messages	No	supplied below
File type and size validation	No	UploadSection.jsx accepts any dropped file unchecked

Program:

File: assignment07_form.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>LipiSutra - Submit a Manuscript</title>
  <style>
    body   { font-family: Georgia, serif; background: #f5ede0; color: #1a0800;
padding: 24px; }
    form   { max-width: 560px; background: #fff; border: 1px solid #8B4513;
             border-radius: 10px; padding: 24px; }
    label  { display: block; margin-top: 14px; font-weight: bold; font-size: 14px; }
    input, select, textarea {
             width: 100%; padding: 8px; margin-top: 4px;
             border: 1px solid #c9b79a; border-radius: 6px; font-size: 14px; }
    .error { color: #b00020; font-size: 12px; min-height: 16px; display: block; }
    .ok    { border-color: #2e7d32; }
    .bad   { border-color: #b00020; background: #fff5f5; }
    button { margin-top: 18px; padding: 10px 20px; border: 0; border-radius: 6px;
             background: #6B2D0E; color: #fff8f0; font-size: 15px; cursor: pointer; }
    button:disabled { background: #b9a893; cursor: not-allowed; }
    #counter { font-size: 12px; color: #5c3317; }
    #banner  { padding: 10px; border-radius: 6px; margin-bottom: 14px; display:
none; }
  </style>
</head>
<body>

  <h1>Submit a Manuscript for Decoding</h1>

  <!-- novalidate turns off the browser bubbles so we can render our own messages -->
  <form id="submitForm" novalidate>

    <div id="banner"></div>

    <label for="name">Contributor Name</label>
    <input id="name" name="name" type="text" required minlength="3" maxlength="40"
           pattern="^[A-Za-z ]{3,40}$" placeholder="Prachi Satbhai">
    <span class="error" id="err-name"></span>

    <label for="email">Email Address</label>
    <input id="email" name="email" type="email" required
placeholder="you@example.com">
    <span class="error" id="err-email"></span>
```

```
    <label for="mobile">Mobile Number</label>
    <input id="mobile" name="mobile" type="text" required pattern="^[6-9][0-9]{9}$"
           placeholder="9876543210">
    <span class="error" id="err-mobile"></span>

    <label for="accession">Accession Number</label>
    <input id="accession" name="accession" type="text" required pattern="^LS-[0-9]
{4}$"
           placeholder="LS-0147">
    <span class="error" id="err-accession"></span>

    <label for="script">Expected Script</label>
    <select id="script" name="script" required>
      <option value="">-- select --</option>
      <option>Modi</option><option>Devanagari</option>
      <option>Brahmi</option><option>Sharada</option>
    </select>
    <span class="error" id="err-script"></span>

    <label for="year">Estimated Year (CE)</label>
    <input id="year" name="year" type="number" required min="-500" max="2026"
placeholder="1712">
    <span class="error" id="err-year"></span>

    <label for="notes">Provenance Notes</label>
    <textarea id="notes" name="notes" rows="4" maxlength="200"
              placeholder="Where was this document found?"></textarea>
    <span id="counter">0 / 200</span>
    <span class="error" id="err-notes"></span>

    <label for="artifact">Artifact Image (JPG or PNG, max 5 MB)</label>
    <input id="artifact" name="artifact" type="file" accept="image/png, image/jpeg"
required>
    <span class="error" id="err-artifact"></span>

    <img id="preview" alt="" style="display:none; max-width:240px; margin-top:10px;
         border:1px solid #8B4513; border-radius:6px;">

    <button type="submit" id="submitBtn">Submit for Decoding</button>
  </form>

  <script src="assignment07_validation.js" defer></script>
</body>
</html>
```

File: assignment07_validation.js

```
"use strict";

const form   = document.getElementById("submitForm");
const banner = document.getElementById("banner");

// ---------- rule table : one entry per field ----------
const RULES = {
  name: {
    test: v => /^[A-Za-z ]{3,40}$/.test(v.trim()),
    msg:  "Enter 3 to 40 letters only (no digits or symbols)."
  },
  email: {
    test: v => /^[^@\s]+@[^@\s]+\.[A-Za-z]{2,}$/.test(v.trim()),
    msg:  "Enter a valid email address, for example you@example.com."
  },
  mobile: {
    test: v => /^[6-9][0-9]{9}$/.test(v.trim()),
    msg:  "Enter a 10-digit mobile number starting with 6, 7, 8 or 9."
  },
  accession: {
    test: v => /^LS-[0-9]{4}$/.test(v.trim().toUpperCase()),
```

```
    msg:  "Accession number must look like LS-0147."
  },
  script: {
    test: v => v !== "",
    msg:  "Select the expected script."
  },
  year: {
    test: v => v !== "" && Number(v) >= -500 && Number(v) <= 2026,
    msg:  "Year must be between -500 and 2026."
  },
  notes: {
    test: v => v.length <= 200,
    msg:  "Provenance notes cannot exceed 200 characters."
  }
};

const MAX_BYTES    = 5 * 1024 * 1024;              // 5 MB
const ALLOWED_MIME = ["image/jpeg", "image/png"];

// ---------- show / clear a field-level message ----------
function setError(id, message) {
  const field = document.getElementById(id);
  const slot  = document.getElementById("err-" + id);
  slot.textContent = message;
  field.classList.toggle("bad", Boolean(message));
  field.classList.toggle("ok", !message && field.value !== "");
  // feed the message into the Constraint Validation API as well
  field.setCustomValidity(message);
}

// ---------- validate one text / select / number field ----------
function validateField(id) {
  const field = document.getElementById(id);
  const rule  = RULES[id];
  if (!rule) return true;

  if (field.required && field.value.trim() === "") {
    setError(id, "This field is required.");
    return false;
  }
  if (field.value.trim() !== "" && !rule.test(field.value)) {
    setError(id, rule.msg);
    return false;
  }
  setError(id, "");
  return true;
}

// ---------- validate the file input : type and size ----------
function validateFile() {
  const input = document.getElementById("artifact");
  const file  = input.files[0];

  if (!file) {
    setError("artifact", "Please choose an artifact image.");
    return false;
  }
  if (!ALLOWED_MIME.includes(file.type)) {
    setError("artifact", `Unsupported type "${file.type}". Upload a JPG or PNG.`);
    return false;
  }
  if (file.size > MAX_BYTES) {
    const mb = (file.size / 1024 / 1024).toFixed(1);
    setError("artifact", `File is ${mb} MB. The limit is 5 MB.`);
    return false;
  }
  setError("artifact", "");
  return true;
}
```

```
// ---------- live validation on blur and on input ----------
Object.keys(RULES).forEach(id => {
  const field = document.getElementById(id);
  field.addEventListener("blur",  () => validateField(id));
  field.addEventListener("input", () => {
    if (field.classList.contains("bad")) validateField(id);   // clear as soon as
fixed
    updateSubmitState();
  });
});

// ---------- live character counter ----------
const notes   = document.getElementById("notes");
const counter = document.getElementById("counter");
notes.addEventListener("input", () => {
  counter.textContent = `${notes.value.length} / 200`;
  counter.style.color = notes.value.length > 180 ? "#b00020" : "#5c3317";
});

// ---------- image preview + file validation on change ----------
const artifact = document.getElementById("artifact");
const preview  = document.getElementById("preview");
artifact.addEventListener("change", () => {
  if (!validateFile()) {
    preview.style.display = "none";
    updateSubmitState();
    return;
  }
  preview.src = URL.createObjectURL(artifact.files[0]);
  preview.style.display = "block";
  updateSubmitState();
});

// ---------- enable the submit button only when everything is valid ----------
function updateSubmitState() {
  const allText = Object.keys(RULES).every(id => {
    const f = document.getElementById(id);
    return !f.required || f.value.trim() !== "";
  });
  document.getElementById("submitBtn").disabled =
    !allText || artifact.files.length === 0;
}

// ---------- submit handler ----------
form.addEventListener("submit", (e) => {
  e.preventDefault();                       // stop the default page reload

  // validate EVERY field, do not stop at the first failure
  const results = Object.keys(RULES).map(validateField);
  const fileOk  = validateFile();
  const valid   = results.every(Boolean) && fileOk;

  banner.style.display = "block";
  if (!valid) {
    const failed = results.filter(r => !r).length + (fileOk ? 0 : 1);
    banner.style.background = "#ffe5e5";
    banner.style.color      = "#b00020";
    banner.textContent      = `${failed} field(s) need attention. The values you typed
were kept.`;
    // move focus to the first invalid control
    form.querySelector(".bad")?.focus();
    return;
  }

  banner.style.background = "#e6f4ea";
  banner.style.color      = "#1e6b33";
  banner.textContent      = "Validation passed. Queuing the manuscript for
decoding...";
```

```
  const payload = Object.fromEntries(new FormData(form).entries());
  delete payload.artifact;                  // the binary is uploaded separately
  console.log("Validated payload:", payload);
});

updateSubmitState();
```

Output:

- The Submit button starts disabled and becomes enabled only once every required field has a value and a file has been chosen.
- Typing "Pr" into the name field and tabbing away shows "Enter 3 to 40 letters only (no digits or symbols)." and turns the field border red; adding a third letter clears the message on the next keystroke.
- Entering 1234567890 as the mobile number is rejected, while 9876543210 is accepted.
- Entering LS-147 is rejected; LS-0147 is accepted.
- Choosing a .pdf file reports "Unsupported type "application/pdf". Upload a JPG or PNG."; choosing a 7 MB photo reports "File is 7.0 MB. The limit is 5 MB."
- A valid JPG shows a thumbnail preview immediately below the file input.
- Submitting an incomplete form shows the red banner, keeps every typed value and moves focus to the first invalid field; submitting a complete form shows the green banner and logs the validated payload object in the console.

Conclusion:

In this assignment, we implemented frontend functionality and complete client-side form validation. We combined HTML5 constraint attributes with custom regular-expression rules, used the Constraint Validation API through setCustomValidity, rendered inline per-field error messages, validated a file input for MIME type and size, and added supporting behaviour such as a live character counter, an image preview and a submit button gated on form validity. We also established that the LipiSutra project currently has controlled form inputs but no validation layer, so this work extends it rather than repeating it.

References:

7. Mozilla Developer Network (MDN) Web Docs - JavaScript Guide, https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
8. W3Schools - JavaScript Tutorial, https://www.w3schools.com/js/
9. ECMAScript Language Specification, https://tc39.es/ecma262/
10. MDN Web Docs - Client-side form validation, https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation
11. MDN Web Docs - Constraint Validation API, https://developer.mozilla.org/en-US/docs/Web/HTML/Constraint_validation